Evolução da Qualidade Estrutural de Projetos Spring: transição para Spring Boot e evidências de assistência por IA

Gabriel Ferreira[1], **Guilherme Augusto**[1]

[1]Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

{gabriel.amaral.1447804,gajsouza}@sga.pucminas.br

Abstract. *This longitudinal study evaluates the evolution of software structural quality through Chidamber and Kemerer (CK) metrics across four technological eras, E1 (2009–2013), E2 (2014–2017), E3 (2018–2021), and E4 (2022–2026), and analyzes structural quality differences in projects with artificial intelligence (AI) assistance in the Spring ecosystem. From 785 Java candidates created through 2017 and active since 2022, RepoReaper-inspired engineering curation without star thresholds approved 607 repositories, of which 470 had valid CK extractions in all eras of their group (1,427 snapshots: 17 pioneer across E1–E4 and 453 main across E2–E4). Across eras, the same systems showed monotonic growth in coupling (CBO) and response (RFC), slight declines in complexity (WMC) and lack of cohesion (LCOM*), and stable inheritance. In E4, projects with detected AI traces ($n = 75$) had higher coupling and complexity means than those without ($n = 395$), while keeping comparable cohesion and inheritance in codebases about fifteen times larger in lines of code, characterizing structural differences without causal attribution.*

Resumo. *Este estudo longitudinal avalia a evolução da qualidade estrutural de* software *por meio das métricas de Chidamber e Kemerer (CK) ao longo de quatro eras tecnológicas, E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022–2026), e analisa diferenças de qualidade estrutural em projetos com assistência por inteligência artificial (IA) no ecossistema Spring. De 785 candidatos Java criados até 2017 e ativos desde 2022, uma curadoria de engenharia inspirada no RepoReaper (sem corte por estrelas) aprovou 607 projetos, dos quais 470 tiveram extração CK válida em todas as eras de seu grupo (1.427* snapshots*: 17 pioneiros de E1 a E4 e 453 principais de E2 a E4). Entre as eras, os mesmos sistemas exibiram alta monotônica em acoplamento (CBO) e resposta (RFC), leve redução em complexidade (WMC) e falta de coesão (LCOM*) e herança estável. Em E4, projetos com vestígios de IA ($n = 75$) tiveram médias maiores de acoplamento e complexidade que os demais ($n = 395$), mantendo coesão e herança próximas em bases cerca de quinze vezes maiores em linhas de código, caracterizando diferenças estruturais sem inferência causal.*

Bacharelado em Engenharia de *Software* (PUC Minas)
Trabalho de Conclusão de Curso II (TCC II)
Orientador: Prof. Cleiton Silva Tavares (cleitontavares@pucminas.br)
Belo Horizonte, novembro de 2026.

1. Introdução

A qualidade interna de *software* descreve propriedades estruturais do código-fonte, como modularidade, coesão e acoplamento, investigáveis por análise estática [ISO/IEC 2011, Pressman and Maxim 2020]. Em sistemas orientados a objetos, a suíte de Chidamber e Kemerer oferece indicadores clássicos para caracterizar essas dimensões em nível de classe [Chidamber and Kemerer 1994]. Tais indicadores não constituem, isoladamente, uma nota única de maturidade: sua interpretação depende do porte, do domínio arquitetural e da organização modular do projeto [El Emam et al. 2001, Aniche et al. 2016].

O ecossistema Spring permite observar a evolução estrutural de aplicações Java ao longo de diferentes ciclos tecnológicos. Sistemas iniciados antes da consolidação do Spring Boot atravessaram mudanças de configuração, gerenciamento de dependências, práticas de verificação e ferramentas de apoio à codificação. Como o Spring Boot 1.0 estável foi lançado em 1º de abril de 2014 [Spring 2014], o intervalo de 2009 a 2013 representa o contexto predominante do Spring tradicional, ao passo que os anos seguintes marcam a difusão da autoconfiguração. A presença atual de dependências do Boot em um repositório criado em 2010 não implica que esse *framework* já integrasse sua arquitetura inicial.

Cuidado análogo aplica-se à assistência por inteligência artificial generativa. Repositórios longevos podem exibir hoje artefatos de agentes introduzidos apenas em versões recentes: a data de criação do repositório não localiza a adoção dessas ferramentas, e recortes de calendário não separam de modo absoluto código humano e assistido. O GitHub Copilot, por exemplo, já operava em prévia técnica em junho de 2021 [GitHub 2021]. Vestígios em arquivos de configuração, identidades automatizadas e mensagens de *commit* tornam observável parte dessa adoção, sem capturar todo uso silencioso nem quantificar a fração de código gerada [Robbes et al. 2026].

A questão central desta pesquisa é: **como a qualidade estrutural de *software*, avaliada por métricas CK, evoluiu ao longo das eras tecnológicas em projetos do ecossistema Spring, e quais diferenças estruturais distinguem os projetos com evidências de assistência por IA?** O foco principal recai sobre a medição longitudinal dos mesmos repositórios que atravessaram essas eras (criados até 2017 e ativos a partir de 2022) e sobre a comparação dos indicadores CK de qualidade na era contemporânea. Para evitar distorções de popularidade, a seleção substitui cortes por contagem de estrelas por uma curadoria de práticas de engenharia inspirada no RepoReaper [Munaiah et al. 2017]. Os projetos pioneiros (criados até 2013) permitem acompanhar trajetórias iniciadas antes do lançamento do Spring Boot, enquanto o grupo principal (2014–2017) retrata sistemas nascidos após sua introdução, isolando a evolução interna de cada código de flutuações de composição amostral.

O objetivo geral consiste em avaliar longitudinalmente a qualidade estrutural dos repositórios selecionados por meio de métricas CK ao longo das eras tecnológicas de que participaram, a partir da última versão disponível em cada período (e do *commit* `HEAD` da *branch* principal na coleta da era recente), comparando também o perfil de qualidade estrutural dos sistemas com e sem evidências detectadas de IA. Os objetivos específicos são: (i) mensurar e descrever as trajetórias de acoplamento, resposta, complexidade, tamanho de métodos, herança e coesão nos mesmos sistemas ao longo das eras; (ii) analisar o comportamento das métricas CK diante de transições declaradas do Spring e do Spring

Boot; (iii) comparar os indicadores de qualidade estrutural entre projetos com e sem evidências detectadas de assistência por IA na era recente; e (iv) contextualizar as métricas CK segundo a curadoria sem filtro de estrelas, as faixas de popularidade e o porte físico do código. O paradigma *Goal-Question-Metric* (GQM) vincula essas questões às sete medidas extraídas pela ferramenta CK [Basili et al. 1994, Aniche 2015].

O estudo delimita suas conclusões ao conjunto observado, sem tomá-lo como censo de toda a comunidade Spring. Idade de repositório, adoção de *framework* ou retração numérica de uma métrica não equivalem a maturidade, e o escopo estritamente estrutural não avalia produtividade, corretude funcional ou desempenho em execução. Na mineração de repositórios, variações de porte, equipe, domínio e recência atuam como explicações concorrentes para deslocamentos nas métricas [Hassan 2008, Kalliamvakou et al. 2016], impedindo atribuir diferenças entre períodos isoladamente ao *framework* ou à IA.

O restante deste artigo organiza-se da seguinte forma: a Seção 2 expõe a fundamentação teórica; a Seção 3 discute os trabalhos relacionados; a Seção 4 detalha o protocolo metodológico, a curadoria e o critério de completude longitudinal; a Seção 5 apresenta e discute os resultados sobre os 470 repositórios válidos finais (1.427 *snapshots*) e as ameaças à validade; e a Seção 6 sintetiza as conclusões e direções futuras.

2. Fundamentação Teórica

Esta seção apresenta os conceitos que sustentam a investigação: qualidade interna de *software* e métricas orientadas a objetos, o paradigma *Goal-Question-Metric*, a mineração de repositórios históricos e os vestígios observáveis de assistência por inteligência artificial.

2.1. Qualidade interna e métricas orientadas a objetos

A norma ISO/IEC 25010 define qualidade de *software* como o grau em que um sistema satisfaz necessidades explícitas e implícitas [ISO/IEC 2011]. Nesse modelo, a qualidade interna compreende propriedades estruturais do código-fonte aferíveis por análise estática, tais como modularidade, coesão e manutenibilidade. Sistemas com estrutura interna disciplinada tendem a demandar menor esforço evolutivo ao longo de seu ciclo de vida [Pressman and Maxim 2020].

Para mensurar propriedades de projeto em sistemas orientados a objetos, Chidamber e Kemerer propuseram seis indicadores clássicos em nível de classe [Chidamber and Kemerer 1994]: acoplamento entre objetos (*Coupling Between Objects*, CBO), que expressa dependências entre classes; métodos ponderados por classe (*Weighted Methods per Class*, WMC), que soma a complexidade dos métodos; profundidade da árvore de herança (*Depth of Inheritance Tree*, DIT) e número de filhos (*Number of Children*, NOC), que descrevem a estrutura hierárquica; resposta de uma classe (*Response for a Class*, RFC), que dimensiona métodos potencialmente acionáveis; e falta de coesão em métodos (*Lack of Cohesion in Methods*, LCOM), que avalia a disjunção no acesso a atributos. Essas medidas consolidaram-se em estudos empíricos sobre arquitetura, manutenção e propensão a defeitos [Singh et al. 2010, Radjenović et al. 2013, Aniche et al. 2016].

Na operacionalização adotada neste estudo (Seção 4), WMC quantifica complexidade estrutural estática segundo a ferramenta CK [Aniche 2015], sem medir custo com-

putacional em tempo de execução. O CBO corresponde ao acoplamento geral entre tipos resolvidos na árvore sintática, sem distinção entre eixos horizontal e vertical, e a coesão é representada pela variante normalizada `lcom*`, em vez do `lcom` não normalizado. As seis métricas de classe são acompanhadas pela média de linhas de código por método (LOC/método) como indicador complementar de granularidade, sem tratá-la como integrante original da suíte CK.

A leitura dessas medidas exige interpretação conjunta e contextualizada. Alterações em DIT, NOC, WMC ou CBO descrevem reconfigurações estruturais, mas não provam isoladamente melhoria arquitetural nem violação de princípios de projeto, pois sofrem influência direta do papel das classes e do porte físico do sistema [El Emam et al. 2001, Aniche et al. 2016].

2.2. Goal-Question-Metric

O paradigma *Goal-Question-Metric* (GQM) estrutura programas de medição em três níveis hierárquicos: objetivo conceitual (*goal*), questões operacionais (*questions*) e indicadores quantitativos (*metrics*) [Basili et al. 1994]. Ao subordinar a coleta de métricas a perguntas explicitamente delimitadas, o GQM previne extrações desconectadas do problema investigado. Neste trabalho, o paradigma conecta o objetivo de caracterizar a evolução estrutural de sistemas Spring às dimensões de acoplamento, complexidade, herança, coesão e porte especificadas na Seção 4.

2.3. Mineração de Repositórios de Software

A Mineração de Repositórios de *Software* (*Mining Software Repositories*, MSR) extrai evidências empíricas de artefatos versionados, tais como código-fonte, grafos de *commits* e metadados, para investigar padrões evolutivos reais [Hassan 2008]. Em plataformas como o GitHub, estudos metodológicos demonstram a necessidade de filtrar *forks*, distinguir data de cadastro do repositório e idade efetiva de cada versão, e substituir limiares de popularidade social (como contagem de estrelas) por critérios objetivos de práticas de engenharia, evitando descartar projetos corporativos ou independentes de menor exposição pública [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016, Munaiah et al. 2017].

2.4. Inteligência artificial generativa

Neste estudo, inteligência artificial generativa refere-se a modelos de linguagem de grande porte treinados em *corpora* de texto e código, capazes de sintetizar ou completar programas a partir de contexto e de *prompts* [Hou et al. 2024]. Assistentes como GitHub Copilot, ChatGPT, Claude Code e Cursor exemplificam modalidades distintas de suporte introduzidas em momentos variados. Embora a era E4 (iniciada em 2022) demarque a difusão comercial dessas soluções, pertencer ao período não atesta uso de IA, assim como versões anteriores não o excluem por completo.

Experimentos controlados sobre tempo de tarefa e correção funcional motivam examinar o fenômeno, mas avaliam construtos distintos das métricas CK de repositórios completos [Peng et al. 2023, Moradi Dakhel et al. 2023]. Além disso, agentes autônomos tendem a deixar artefatos explícitos, como arquivos de instruções e registros de coautoria em *commits*, com frequência superior à de assistentes de completamento *inline* [Robbes et al. 2026]. Assim, os vestígios observáveis capturam apenas a parcela rastreável dessa adoção.

3. Trabalhos Relacionados

A revisão da literatura organiza-se em quatro eixos complementares: evolução longitudinal de sistemas por mineração de repositórios, curadoria de projetos de engenharia no GitHub, interpretação contextual de métricas orientadas a objetos e detecção de assistência por inteligência artificial.

3.1. Evolução de sistemas e mineração de repositórios

Godfrey e Tu acompanharam 96 versões sequenciais do núcleo Linux e demonstraram que a compreensão de trajetórias evolutivas requer inspecionar a organização interna dos subsistemas, além do crescimento agregado em tamanho [Godfrey and Tu 2000]. A convergência com este TCC é metodológica: observar sucessivos *snapshots* dos mesmos projetos revela inflexões internas invisíveis em recortes transversais estáticos, embora o objeto e os indicadores aqui adotados concentrem-se em aplicações Java do ecossistema Spring medidas pela suíte CK.

A disponibilidade de repositórios públicos não assegura, por si só, validade amostral [Hassan 2008]. Kalliamvakou et al. sistematizaram ameaças recorrentes na mineração do GitHub, incluindo repositórios inativos, espelhos, projetos pessoais efêmeros e discrepâncias entre metadados da plataforma e histórico Git efetivo [Kalliamvakou et al. 2014, Kalliamvakou et al. 2016]. Esses alertas fundamentam a separação, neste estudo, entre registros descobertos via API, históricos recuperáveis por era e projetos com completude longitudinal, reconhecendo o viés de sobrevivência inerente à exigência de atividade contemporânea.

3.2. Curadoria de repositórios de software

Munaiah et al. propuseram um arcabouço para identificar projetos de *software* de engenharia em meio a repositórios públicos do GitHub e implementaram a ferramenta `reaper`, baseada em sinais objetivos de práticas de desenvolvimento [Munaiah et al. 2017]. Os autores evidenciaram que cortes por número de estrelas constituem aproximações inadequadas de relevância técnica: limiares de popularidade excluem ampla fração de sistemas reais mantidos por equipes independentes ou organizações com baixa visibilidade social, enquanto retêm repositórios populares de finalidade didática ou documental, diagnóstico convergente com análises anteriores da plataforma [Kalliamvakou et al. 2016].

Neste TCC, incorpora-se diretamente esse princípio ao substituir o piso de estrelas por uma curadoria multidimensional local em quatro critérios (Seção 4.3), adaptando o filtro lexical de projetos brinquedo às convenções de nomenclatura do ecossistema Spring Boot. O artigo de referência fundamenta o conceito de curadoria por práticas de engenharia [Munaiah et al. 2017], ao passo que as regras heurísticas e adaptações lexicais constituem decisões operacionais próprias deste protocolo.

3.3. Interpretação contextual das métricas

A abordagem SATT propõe limiares de métricas calibrados segundo papéis arquiteturais específicos, demonstrando em sistemas MVC e Android que variações agregadas refletem diferenças legítimas de responsabilidade entre classes [Aniche et al. 2016]. Em vez de classificar repositórios por notas normativas, o presente trabalho aplica o mesmo extrator

estático ao longo das eras e interpreta os deslocamentos médios à luz da escala e da composição modular.

No domínio de injeção de dependências, Sun et al. desenvolveram métricas ponderadas para distinguir acoplamentos rígidos e flexíveis associados à manutenibilidade em Java [Sun et al. 2022]. O resultado evidencia que mecanismos típicos do Spring interagem com indicadores clássicos de dependência. Este estudo preserva o CBO geral implementado no CK para assegurar comparabilidade histórica direta, documentando explicitamente a variante empregada conforme recomendado em avaliações comparativas de métricas de acoplamento [Child et al. 2019].

3.4. Assistência por IA e identificação de vestígios

A difusão de Modelos de Linguagem de Grande Porte (LLMs) na engenharia de *software* reconfigurou fluxos de codificação a partir de 2022 [Hou et al. 2024]. Estudos experimentais sobre o GitHub Copilot e assistentes correlatos investigaram tempo de conclusão de tarefas, carga cognitiva de revisão e qualidade de trechos gerados [Peng et al. 2023, Moradi Dakhel et al. 2023, Yetistiren et al. 2023, Mozannar et al. 2024], enquanto o arcabouço EvalPlus ampliou testes automatizados para aferir a correção funcional de soluções sintetizadas [Liu et al. 2023]. Embora motivem o recorte temporal da era E4, esses experimentos avaliam tarefas isoladas e não predizem diretamente o comportamento das métricas CK em repositórios longevos.

Para distinguir automação em históricos de versão, a abordagem BIMAN combina padrões de identidade, mensagens e atividade na detecção de *bots* de *commit* [Dey et al. 2020]. Contudo, automações convencionais não equivalem a modelos generativos, e desenvolvedores frequentemente utilizam assistentes sob suas próprias credenciais Git. Em levantamento recente sobre 128.018 projetos do GitHub, Robbes et al. mapearam a adoção de agentes de codificação por meio de arquivos dedicados de configuração e *trailers* de coautoria, diferenciando-os do completamento silencioso [Robbes et al. 2026]. O detector deste TCC adapta localmente a inspeção temporal desses artefatos no histórico Git até cada *snapshot*, sem herdar estimativas externas de precisão nem inferir a proporção de código assistido.

Em síntese, a contribuição deste trabalho reside em avaliar longitudinalmente a qualidade estrutural dos mesmos projetos Spring pelas métricas CK ao longo das eras tecnológicas e comparar os indicadores estruturais de projetos com e sem vestígios rastreáveis de IA, apoiando-se em curadoria local sem viés de estrelas.

4. Materiais e Métodos

Este estudo adota delineamento observacional, longitudinal e descritivo sobre repositórios públicos do ecossistema Spring no GitHub. A unidade acompanhada é o repositório, e a unidade de medição corresponde ao último *snapshot* válido na linha principal dentro de cada era (em E4, o *commit* `HEAD` da *branch* principal coletado em outubro de 2026). As análises caracterizam trajetórias históricas e associações empíricas, sem reivindicar identificação causal isolada para o Spring Boot ou para ferramentas de inteligência artificial.

4.1. Base de origem, ausência de filtro de estrelas e recorte de candidatos

A população inicial compreende os 95.894 registros únicos consolidados em `population.csv` (execução `20260920_java`), coletados via API REST de busca

do GitHub mediante a consulta `topic:spring-boot fork:true`, particionada por janelas de data de criação e, apenas quando necessário para contornar o teto técnico de mil resultados por requisição da API, subdividida por faixas de estrelas sem descarte de registros.

Sobre os 95.894 registros aplicaram-se filtros sequenciais e determinísticos de metadados, contabilizando cada repositório na primeira condição de rejeição: linguagem principal ausente ou distinta de Java (23.298), *forks* (1.089), espelhos (0), arquivados (1.607), desabilitados (0), ausência de atividade a partir da abertura da era E4 em `2022-01-01T00:00:00Z` (16.615), *templates* (323), repositórios vazios ou sem *branch* padrão (3) e listas ou exercícios retidos por triagem lexical preliminar estritamente escolar (540). As 43.475 exclusões resultaram em 52.419 repositórios Java ativos em E4. Em seguida, o recorte histórico de criação até o final de 2017 em UTC (`created_at < 2018-01-01T00:00:00Z`) descartou 51.634 projetos recentes e definiu os 785 repositórios candidatos do estudo: 22 criados até 2013 e 763 criados entre 2014 e 2017.

Diferentemente de levantamentos que impõem cortes arbitrários de popularidade (como o limiar usual de dez estrelas), este protocolo não aplica filtro por número de estrelas. Contadores de estrelas refletem exposição social ou difusão didática, e não práticas de engenharia de *software* [Kalliamvakou et al. 2016, Munaiah et al. 2017]. Entre os 785 candidatos, 389 (49,6%) possuem menos de dez estrelas (sendo 156 com zero e 73 com uma estrela) e 396 (50,4%) possuem dez ou mais estrelas. Excluir projetos com menos de dez estrelas descartaria metade dos candidatos históricos, como microsserviços organizacionais e sistemas mantidos por equipes independentes; por isso, a relevância técnica é aferida pela curadoria multidimensional descrita na Seção 4.3.

4.2. Janela de atividade na era E4 e fundamentação temporal

O critério de atividade contemporânea adota `pushed_at >= 2022-01-01T00:00:00Z`, alinhando a exigência de sobrevivência operacional exatamente ao início da era E4 (2022–2026), marco da disseminação de LLMs, assistentes de código e agentes de programação [Hou et al. 2024, Mozannar et al. 2024, Robbes et al. 2026]. A adoção de janeiro de 2022, em substituição a um corte móvel de três anos contado de setembro de 2023 (que reteria 540 candidatos), preserva 245 sistemas Java longevos criados até 2017 que permaneceram ativos durante o ciclo da assistência generativa, em consonância com janelas plurianuais empregadas na mineração de projetos Java [Oliveira et al. 2021]. Registra-se como limitação dos metadados que `pushed_at` atua como aproximação de atividade (podendo refletir envio de *tags* ou atualizações em ramos secundários, e não *commits* contínuos na *branch* padrão) e que o descarte de arquivados ou inativos antes de 2022 introduz viés de sobrevivência.

4.3. Higienização local inspirada no RepoReaper e adaptação ao Spring Boot

Para selecionar projetos com práticas verificáveis de engenharia e descartar exercícios acadêmicos, rascunhos efêmeros ou depósitos não estruturados, implementou-se em ambiente local o instrumento determinístico `reporeaper_curation.py`, fundamentado nas dimensões conceituais do arcabouço RepoReaper [Munaiah et al. 2017] e nas diretrizes de triagem de repositórios do GitHub [Kalliamvakou et al. 2016]. A aprovação

exige atendimento simultâneo a quatro critérios operacionais (C1 + C2 Ajustado + C3 + C4):

1. **Critério C1 (Finalidade não didática):** exclusão de repositórios voltados a disciplinas, cursos ou exercícios, identificados pelos 14 vocábulos `course`, `homework`, `assignment`, `learning`, `tutorial`, `aula`, `curso`, `exercicio`, `exercicios`, `treinamento`, `udemy`, `mooc`, `exercism` e `lecture` em nome, descrição ou tópicos, preservando compostos técnicos de domínio (`machine-learning` e `deep-learning`).
2. **Critério C2 Ajustado (Exclusão de projetos brinquedo adaptada ao Spring Boot):** rejeição de áreas de teste descartáveis e esqueletos identificados por `playground`, `sandbox`, `toy`, `practice`, `boilerplate`, `cheatsheet`, `cheat-sheet` e `slides`, isentando de penalização os termos `demo`, `sample` e `example` (e suas formas plurais).
3. **Critério C3 (Automação de *build* padronizada):** comprovação de descritores formais de compilação e dependências do ecossistema Java (`pom.xml`, `build.gradle` ou `build.gradle.kts`).
4. **Critério C4 (Presença de testes automatizados):** comprovação estrutural de verificação automatizada pela presença do diretório padrão `src/test` ou de classes `*Test.java` e `*Tests.java`.

A isenção de `demo`, `sample` e `example` no Critério C2 decorre de duas convenções técnicas do ecossistema Spring Boot: o gerador oficial Spring Initializr (`start.spring.io`) preenche por padrão o identificador de artefato `demo` (assim como gera classes `*ApplicationTests.java` contempladas em C4), e mantenedores corporativos nomeiam rotineiramente implementações completas de microsserviços, autenticação OAuth2 e mensageria Kafka com `sample` ou `example`. A auditoria empírica confirmou que bloquear esses três radicais descartaria 149 repositórios aprovados nos demais critérios de engenharia (114 deles com histórico CK completo em todas as eras), dotados de *build* Maven/Gradle, suíte de testes e manutenção plurianual. Nos 785 candidatos, observaram-se 765 aprovações em C1 (97,5%), 768 em C2 Ajustado (97,8%), 752 em C3 (95,8%) e 665 em C4 (84,7%); a interseção estrita dos quatro critérios aprovou 607 repositórios (77,3%): 20 criados até 2013 e 587 criados entre 2014 e 2017.

4.4. Grupos temporais, seleção de SHAs e completude longitudinal

As eras tecnológicas analisadas são E1 (2009–2013), E2 (2014–2017), E3 (2018–2021) e E4 (2022 até outubro de 2026). A data de criação define o grupo de ingresso, enquanto a data do *commit* posiciona cada medição em sua era efetiva. Em E1, E2 e E3, percorrendo a história pela linha do primeiro pai (`--first-parent`) nos *commits* de *merge* [Git 2026], seleciona-se o último *commit* com data de *committer* contida na era, sem projetar retroativamente código futuro na ausência de *commit* no período. Em E4, fixa-se o SHA do `HEAD` da *branch* principal na coleta; quando a atividade recente capturada por `pushed_at` ocorreu apenas em ramos secundários ou *tags*, o `HEAD` de E4 conserva legitimamente o mesmo SHA observado ao final de E3 (variação nula no par E3–E4).

Para eliminar flutuações de composição amostral entre períodos, adotou-se critério estrito de painel longitudinal balanceado: um repositório integra a amostra analítica final somente quando possui *snapshot* recuperável e extração válida do CK em todas as eras

previstas para seu grupo temporal. Dos 607 repositórios aprovados na curadoria, 137 não cumpriram essa completude (122 por ausência de *commit* na linha principal em alguma era intermediária e 15 por falha de clonagem ou ausência de classes Java mensuráveis em ao menos uma era), saindo da síntese sem substituição. A amostra final reúne 470 repositórios válidos (1.427 *snapshots* CK), divididos em dois grupos disjuntos: (1) **Grupo Pioneiro** (criados até 31/12/2013), com 17 repositórios acompanhados continuamente nas quatro eras (E1, E2, E3 e E4; 68 *snapshots*); e (2) **Grupo Principal** (criados entre 2014 e 2017), com 453 repositórios acompanhados continuamente nas três eras (E2, E3 e E4; 1.359 *snapshots*). Assim, todas as comparações entre eras operam exatamente sobre os mesmos identificadores em cada grupo ($n = 17$ e $n = 453$).

4.5. Extração estática CK e medidas contextuais de porte

As métricas estruturais foram extraídas sobre cada árvore recuperada por `checkout` determinístico do SHA com o CK 0.7.1-SNAPSHOT (revisão `8a1ef916`), baseado no Eclipse JDT 3.44.0 (configuração Java 21) sob JVM OpenJDK 17 [Aniche 2015], sem compilação ou execução dos projetos. Calculam-se por *snapshot* as médias de CBO geral, RFC, WMC, DIT, NOC, LCOM* e LOC/método, preservando nos artefatos brutos a separação entre código de produção e de testes.

No extrator, CBO contabiliza tipos distintos acoplados na AST; RFC soma invocações únicas estaticamente resolvidas; WMC agrega pontos de ramificação estrutural por classe; DIT inicia em base unitária na raiz; NOC conta subclasses diretas no *snapshot*; e LCOM* aplica a expressão normalizada $\frac{1}{a}\sum_{i=1}^{a}\frac{m-\mu(A_i)}{m}$, onde a indica atributos, m métodos e $\mu(A_i)$ métodos que acessam cada atributo. Classes com LCOM* matematicamente indefinido são omitidas apenas do denominador dessa métrica, sem imputação por zero, assim como falhas de extração jamais são convertidas em zero. Não se impõe piso mínimo de LOC ou de arquivos Java para inclusão; o porte físico entra como variável contextual descritiva, contando cada caminho distinto `.java` uma única vez e somando linhas físicas não vazias e não exclusivamente comentadas sem duplicação por classes internas, em conformidade com diretrizes formais de medição de tamanho [Park 1992].

4.6. Evidências de IA, síntese GQM e reprodutibilidade

A Tabela 1 sintetiza a correspondência entre as quatro questões de pesquisa (Q1–Q4) e os indicadores operacionalizados pelo paradigma GQM.

A detecção local de vestígios de IA inspeciona arquivos e diretórios de configuração de agentes (incluindo GitHub Copilot, Cursor, Claude Code e Codex/OpenAI), entradas correlatas em `.gitignore`, identidades de *bots* de codificação e *trailers* `Co-authored-by` alcançáveis no histórico Git até o SHA avaliado [Robbes et al. 2026]. A ausência desses sinais não prova inexistência de assistência por IA, nem o primeiro registro versionado data o início exato do uso.

A síntese analítica é estritamente descritiva, atribuindo peso idêntico a cada repositório nas médias por período e abstendo-se de testes inferenciais, p-valores ou delta de Cliff. Pela completude longitudinal dos dois grupos, os resultados são reportados em pares de tabelas (médias por era e diferenças exatas entre eras sobre os mesmos sistemas), mantendo todos os manifestos, SHAs, *hashes* e extrações brutas auditáveis localmente.

Tabela 1. Questões de pesquisa e medidas associadas no paradigma GQM.

Q	Questão de pesquisa	Medidas operacionais
Q1	Como a qualidade estrutural (métricas CK) dos mesmos projetos válidos evolui ao longo das eras?	Médias das sete métricas CK por era e diferenças exatas por grupo ($n = 17$ e $n = 453$).
Q2	Como as métricas estruturais comportam-se nas transições declaradas de Spring e Boot?	Dependências Maven/Gradle em cada SHA e variações longitudinais de CBO.
Q3	Quais diferenças de qualidade estrutural (métricas CK) distinguem projetos com e sem evidências de IA em E4?	Rastros em arquivos e *commits* até o *snapshot* ($N = 470$); médias CK e porte físico.
Q4	Como a curadoria sem filtro de estrelas, a popularidade e o porte contextualizam as métricas CK?	Taxas nos quatro critérios de curadoria, estratos de estrelas, arquivos Java e LOC.

Fonte: Elaborada pelos autores (2026).

5. Resultados e Discussão

5.1. Amostra final válida e análise por faixas de estrelas

A filtragem de metadados com atividade em E4 e criação até 2017 selecionou 785 candidatos, dos quais 607 superaram simultaneamente os quatro critérios locais de curadoria. Aplicado o requisito de extração válida do CK em todas as eras do respectivo grupo temporal, consolidou-se o painel final de 470 repositórios válidos e 1.427 *snapshots* analisados (Tabela 2): 17 sistemas no grupo pioneiro (2009–2013, medidos de E1 a E4) e 453 no grupo principal (2014–2017, medidos de E2 a E4).

Tabela 2. Síntese da filtragem e composição da amostra longitudinal válida com extração CK completa em todas as eras de cada grupo ($N = 470$ repositórios).

Grupo	Eras exigidas	Candidatos	Curadoria	Válidos (CK)	Snapshots
Pioneiro (2009–2013)	E1, E2, E3 e E4	22	20	17	68
Principal (2014–2017)	E2, E3 e E4	763	587	453	1.359
Total		785	607	470	1.427

Fonte: Elaborada pelos autores (2026).

A substituição do piso de estrelas pela curadoria de práticas de engenharia preservou ampla diversidade de visibilidade na amostra final: 209 repositórios válidos (44,5%) possuem menos de dez estrelas e 261 (55,5%) possuem dez ou mais estrelas. Em E4, a comparação direta entre os projetos com menos de dez estrelas ($n = 209$) e aqueles com dez ou mais estrelas ($n = 261$) apoia empiricamente essa decisão: ambos os estratos apresentam médias muito próximas em nível de classe e método (CBO de 5,483 contra 5,830, LOC/método de 5,568 contra 5,707 linhas, DIT de 1,200 contra 1,254 e LCOM* de 0,232 contra 0,239), distinguindo-se sobretudo na escala física agregada (88,0 arquivos Java e 3.821 LOC nos menos estrelados, ante 543,1 arquivos e 41.054 LOC nos mais populares). Isso indica que repositórios de menor exposição social aprovados na curadoria e ativos entre eras exibem perfil estrutural comparável nas métricas avaliadas, não se reduzindo a artefatos triviais.

5.2. Evolução longitudinal das métricas CK por grupo temporal

Como o painel longitudinal é estritamente balanceado dentro de cada grupo, as tabelas de médias por era e de diferenças entre eras incidem sobre o mesmo conjunto invariável

de repositórios, assegurando correspondência exata entre a variação média individual dos projetos e a diferença entre as médias dos períodos.

No grupo pioneiro ($n = 17$ repositórios criados até 2013), as Tabelas 3 e 4 mostram crescimento monotônico no acoplamento entre classes (CBO: de 4,878 em E1 para 5,394 em E2, 5,626 em E3 e 5,803 em E4, acumulando $+0{,}516$, $+0{,}748$ e $+0{,}925$) e na resposta por classe (RFC: de 7,639 em E1 para 8,387 em E2, 8,972 em E3 e 9,956 em E4, acumulando $+0{,}748$, $+1{,}333$ e $+2{,}317$). Em direção oposta, a complexidade ponderada (WMC) recuou continuamente de 8,094 em E1 para 7,201 em E4 ($-0{,}893$), e a falta de coesão (LCOM*) caiu de 0,298 para 0,249 ($-0{,}049$), sinalizando classes internamente mais coesas apesar da forte expansão física média desses sistemas (de 1.126,5 arquivos Java e 63.102 LOC em E1 para 2.999,6 arquivos e 228.571 LOC em E4). Os indicadores de herança (DIT e NOC) permaneceram estáveis.

Tabela 3. Grupo pioneiro (criados até 2013, $n = 17$): médias das métricas CK dos mesmos 17 repositórios válidos em E1, E2, E3 e E4.

Métrica	E1	E2	E3	E4
CBO	4,878	5,394	5,626	5,803
RFC	7,639	8,387	8,972	9,956
WMC	8,094	8,076	7,936	7,201
LOC/método	6,316	5,957	6,208	6,446
DIT	1,416	1,355	1,376	1,341
NOC	0,083	0,068	0,104	0,081
LCOM*	0,298	0,276	0,277	0,249

Fonte: Elaborada pelos autores (2026).

Tabela 4. Grupo pioneiro (criados até 2013, $n = 17$): diferenças das médias de E1 em relação às eras subsequentes nos mesmos 17 repositórios válidos.

Métrica	(E2 − E1)	(E3 − E1)	(E4 − E1)
CBO	+0,516	+0,748	+0,925
RFC	+0,748	+1,333	+2,317
WMC	−0,019	−0,158	−0,893
LOC/método	−0,359	−0,108	+0,130
DIT	−0,062	−0,040	−0,076
NOC	−0,015	+0,021	−0,002
LCOM*	−0,022	−0,021	−0,049

Fonte: Elaborada pelos autores (2026). Nota: diferenças calculadas antes do arredondamento.

No grupo principal ($n = 453$ repositórios criados entre 2014 e 2017), as Tabelas 5 e 6 consolidam as médias em E2, E3 e E4 e as diferenças nos intervalos adjacentes e acumulado sobre os mesmos 453 sistemas:

- **E2 vs. E3:** o CBO elevou-se em $+0{,}186$ (de 5,405 para 5,591) e o RFC avançou $+0{,}622$ (de 6,603 para 7,224), enquanto o WMC teve leve retração de $-0{,}018$ (de 5,819 para 5,801) e o LCOM* variou $-0{,}002$ (de 0,245 para 0,243), com acréscimo de $+0{,}181$ linhas em LOC/método e estabilidade em DIT ($+0{,}008$) e NOC ($+0{,}006$).
- **E3 vs. E4:** o CBO cresceu mais $+0{,}080$ (atingindo 5,671) e o RFC aumentou $+0{,}363$ (chegando a 7,587); o WMC recuou $-0{,}042$ (para 5,759) e o LCOM* diminuiu $-0{,}007$ (para 0,235), acompanhados de $+0{,}127$ em LOC/método e constância em DIT ($-0{,}003$) e NOC ($-0{,}002$).

- **E2 vs. E4:** no acumulado das três eras, registraram-se altas consistentes em CBO ($+0{,}266$) e RFC ($+0{,}984$), retrações leves em WMC ($-0{,}060$) e LCOM* ($-0{,}009$), aumento de $+0{,}307$ linhas por método e conservação hierárquica (DIT $+0{,}005$; NOC $+0{,}004$), enquanto o porte físico médio passou de 121,7 arquivos e 7.492 LOC em E2 para 185,7 arquivos e 11.703 LOC em E3 e 241,0 arquivos e 16.839 LOC em E4.

Tabela 5. Grupo principal (criados entre 2014 e 2017, $n = 453$): médias das métricas CK dos mesmos 453 repositórios válidos em E2, E3 e E4.

Métrica	E2	E3	E4
CBO	5,405	5,591	5,671
RFC	6,603	7,224	7,587
WMC	5,819	5,801	5,759
LOC/método	5,308	5,489	5,616
DIT	1,221	1,229	1,226
NOC	0,059	0,065	0,063
LCOM*	0,245	0,243	0,235

Fonte: Elaborada pelos autores (2026).

Tabela 6. Grupo principal (criados entre 2014 e 2017, $n = 453$): diferenças das médias entre E2, E3 e E4 nos mesmos 453 repositórios válidos.

Métrica	(E3 − E2)	(E4 − E3)	(E4 − E2)
CBO	+0,186	+0,080	+0,266
RFC	+0,622	+0,363	+0,984
WMC	−0,018	−0,042	−0,060
LOC/método	+0,181	+0,127	+0,307
DIT	+0,008	−0,003	+0,005
NOC	+0,006	−0,002	+0,004
LCOM*	−0,002	−0,007	−0,009

Fonte: Elaborada pelos autores (2026). Nota: diferenças calculadas antes do arredondamento.

A convergência de direção entre o grupo pioneiro ($n = 17$) e o grupo principal ($n = 453$) evidencia que, à medida que esses sistemas evoluem e expandem sua base de código, a colaboração e o acoplamento entre classes (CBO e RFC) aumentam gradualmente, sem degradar a coesão interna dos atributos (LCOM*) nem inflacionar a complexidade média das classes (WMC).

5.3. Qualidade estrutural nas transições para Spring Boot e em projetos com IA

Na inspeção estática de descritores Maven e Gradle entre eras adjacentes, a passagem de ausência para presença declarada de dependências `org.springframework.boot` ocorreu em sete transições E1–E2 (variação média de CBO de $+0{,}585$), 32 transições E2–E3 ($+0{,}541$) e 11 transições E3–E4 ($-0{,}020$). A dispersão de magnitude e sinal entre períodos, aliada à adoção frequentemente restrita a submódulos específicos em arquiteturas multimódulo, indica que a introdução declarada do Boot não impõe isoladamente um deslocamento uniforme às métricas CK.

Na análise das diferenças de qualidade estrutural associadas à assistência por IA nos 470 repositórios válidos em E4, identificaram-se vestígios explícitos em 75 projetos

Tabela 7. E4: médias entre os repositórios válidos finais ($N = 470$) com evidência detectada de IA (A, $n_a = 75$) e sem evidência detectada (B, $n_b = 395$).

Métrica	Média A	Média B	A − B
CBO	6,101	5,595	+0,506
RFC	9,840	7,261	+2,579
WMC	6,960	5,593	+1,366
LOC/método	6,182	5,544	+0,639
DIT	1,301	1,216	+0,085
NOC	0,079	0,061	+0,018
LCOM*	0,251	0,233	+0,018

Fonte: Elaborada pelos autores (2026). Nota: diferenças calculadas antes do arredondamento.

(16,0%) e ausência dos rastros monitorados em 395 projetos (84,0%), conforme a Tabela 7. Nenhum vestígio explícito dos agentes catalogados foi detectado nos *snapshots* de E1 a E3, o que não exclui uso preliminar não rastreável.

Sob a ótica das métricas CK em E4, o grupo com evidência detectada de IA ($n_a = 75$) exibiu médias superiores nas dimensões de acoplamento, resposta e complexidade frente ao grupo sem evidência detectada ($n_b = 395$), destacando-se CBO (6,101 contra 5,595; $+0{,}506$), RFC (9,840 contra 7,261; $+2{,}579$), WMC (6,960 contra 5,593; $+1{,}366$) e LOC/método (6,182 contra 5,544; $+0{,}639$). Em contrapartida, os indicadores de coesão e herança permaneceram muito próximos entre ambos os grupos, com diferenças residuais em LCOM* (0,251 contra 0,233; $+0{,}018$), DIT (1,301 contra 1,216; $+0{,}085$) e NOC (0,079 contra 0,061; $+0{,}018$). O exame contextual mostra que esse aumento de acoplamento e complexidade acompanha uma acentuada diferença de porte físico: os 75 sistemas com vestígios de IA reúnem, em média, 113.864 LOC e 1.341,6 arquivos Java, ante 7.529 LOC e 150,7 arquivos nos 395 demais (razão aproximada de quinze vezes em linhas de código). Dada a sensibilidade das métricas orientadas a objetos ao tamanho do sistema [El Emam et al. 2001], e na ausência de controle multivariado por porte, domínio e equipe, esses contrastes caracterizam o perfil estrutural dos projetos que versionaram artefatos de agentes, sem autorizar inferência de efeito causal direto da IA sobre a qualidade interna.

A trajetória prévia dos mesmos 470 repositórios entre E3 e E4 reforça essa leitura: nos 75 projetos com vestígios de IA em E4, o CBO médio já partia de patamar superior antes da era generativa, passando de 5,858 em E3 para 6,101 em E4 ($+0{,}243$), enquanto nos 395 sem vestígios evoluiu de 5,542 para 5,595 ($+0{,}053$). Sistemas de maior escala e manutenção mais intensa em E4 tendem simultaneamente a exibir maior interconexão entre classes desde E3 e maior propensão à adoção visível de agentes de codificação.

5.4. Ameaças à validade

Construto. O CK analisa o código-fonte estaticamente sem resolver dependências externas compiladas, podendo subestimar heranças ou acoplamentos indiretos. Valores indefinidos de LCOM* são omitidos apenas de seu denominador, sem imputação por zero. O detector de IA captura exclusivamente rastros explícitos versionados, de modo que o estrato sem evidência detectada pode conter uso silencioso de completamento *inline* ou consultas externas. Por fim, os quatro critérios de curadoria constituem operacionalização heurística local inspirada em Munaiah et al. [Munaiah et al. 2017]; em particular, C3 (*build*) e C4 (testes) aferem a estrutura consolidada do repositório auditado, podendo não

estar presentes nas eras iniciais, e a isenção de `demo`/`sample`/`example` em C2 não substitui auditoria qualitativa manual.

Interna. Deslocamentos temporais e contrastes entre estratos de IA coexistem com forte expansão física (LOC e arquivos), refatorações, mudanças de equipe e variações na recência efetiva do `HEAD` em E4 (incluindo projetos cuja atividade recente ocorreu fora da *branch* principal). Como não há piso mínimo de porte, microsserviços enxutos e plataformas extensas coexistem nas médias, exigindo leitura conjunta com o tamanho contextual e com as diferenças pareadas sobre os mesmos sistemas.

Externa. A descoberta apoia-se no tópico `spring-boot` atualmente atribuído no GitHub, em linguagem principal Java, na exclusão de arquivados, em `pushed_at >= 2022-01-01` e na completude CK em todas as eras do grupo, introduzindo viés de sobrevivência e continuidade em favor de projetos longevos. Ademais, a dimensão reduzida do grupo pioneiro ($n = 17$) impede generalização para todo o universo histórico de sistemas Spring tradicionais ou proprietários.

Conclusão. A análise restringe-se à estatística descritiva, sem testes de hipótese, p-valores ou tamanhos de efeito inferenciais. Como os mesmos repositórios são medidos repetidamente ao longo das eras, as observações não constituem amostras independentes; a consistência descritiva apoia-se na eliminação integral de atrito amostral entre períodos dentro de cada grupo ($n = 17$ em E1–E4 e $n = 453$ em E2–E4).

6. Conclusão

Este trabalho avaliou a evolução da qualidade estrutural de *software* com base nas métricas CK ao longo das eras tecnológicas E1 a E4 e analisou as diferenças estruturais entre projetos com e sem evidências detectadas de assistência por IA no ecossistema Spring. O painel longitudinal acompanhou 470 repositórios Java válidos (1.427 *snapshots* CK), selecionados a partir de 785 candidatos por curadoria local de práticas de engenharia (607 aprovados) e medição completa em todas as eras previstas para cada grupo temporal (17 pioneiros em E1–E4 e 453 principais em E2–E4).

Objetivo específico (i): evolução da qualidade estrutural pelas métricas CK. Nos mesmos sistemas medidos continuamente entre as eras, o acoplamento entre classes (CBO) e o conjunto de resposta (RFC) cresceram de maneira monotônica: $+0{,}925$ em CBO e $+2{,}317$ em RFC nos 17 projetos pioneiros entre E1 e E4, e $+0{,}266$ em CBO e $+0{,}984$ em RFC nos 453 projetos principais entre E2 e E4. Em contrapartida, a complexidade ponderada por classe (WMC) recuou ($-0{,}893$ no grupo pioneiro e $-0{,}060$ no principal) e a falta de coesão (LCOM*) diminuiu ($-0{,}049$ e $-0{,}009$), com estabilidade em herança (DIT e NOC). Os resultados mostram que a expansão física dos projetos ao longo das eras eleva a colaboração entre componentes sem degradar a coesão interna nem inflacionar a complexidade média das classes.

Objetivo específico (ii): transições de Spring e Spring Boot. As transições declaradas para dependências `org.springframework.boot` em descritores Maven e Gradle exibiram variações heterogêneas de CBO entre eras adjacentes ($+0{,}585$ em E1–E2, $+0{,}541$ em E2–E3 e $-0{,}020$ em E3–E4), indicando que a adoção do *framework* interage com a organização modular de cada sistema sem determinar isoladamente o comportamento das métricas CK.

Objetivo específico (iii): diferenças de qualidade em projetos com IA. Em E4, os 75 repositórios válidos com vestígios explícitos de agentes de IA apresentaram médias superiores de acoplamento e complexidade (CBO, RFC, WMC e LOC/método) frente aos 395 sem vestígios detectados, preservando patamares próximos de coesão (LCOM*) e herança (DIT e NOC). Esses 75 sistemas, contudo, concentraram bases de código quinze vezes maiores em LOC médio (113.864 contra 7.529 LOC) e já partiam de acoplamento mais elevado em E3 (CBO de 5,858 contra 5,542), demonstrando que a adoção rastreável de IA concentra-se em sistemas mais extensos e ativos, sem sustentar atribuição causal direta sobre a estrutura interna.

Objetivo específico (iv): curadoria, estrelas e porte. O painel balanceado eliminou variações de composição amostral entre eras e evidenciou que repositórios com menos de dez estrelas validados pela curadoria (44,5% da amostra final) mantêm coesão, acoplamento, herança e granularidade de métodos equivalentes aos de sistemas populares, respaldando critérios objetivos de engenharia em substituição a cortes por estrelas.

Como contribuição central, o estudo entrega uma caracterização longitudinal auditável da qualidade estrutural CK de projetos Spring através de quatro eras tecnológicas e um diagnóstico empírico das diferenças estruturais em sistemas com evidências de IA. Respeitados os limites observacionais detalhados na Seção 5.4, trabalhos futuros podem modelar estereótipos arquiteturais do Spring sobre o acoplamento, aplicar controles regressivos multivariados por porte e investigar impactos de assistência por IA sobre métricas CK no nível de método e de *pull request*.

Referências

Aniche, M. (2015). Java Code Metrics Calculator (CK). `https://github.com/mauricioaniche/ck`. Revisão de desenvolvimento 0.7.1-SNAPSHOT, commit 8a1ef916d597d33ad619d748b616bb7dbc3f0c36; adaptador local com fórmulas preservadas.

Aniche, M., Treude, C., Zaidman, A., van Deursen, A., and Gerosa, M. A. (2016). SATT: Tailoring code metric thresholds for different software architectures. In *2016 IEEE 16th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 41–50. IEEE.

Basili, V. R., Caldiera, G., and Rombach, H. D. (1994). The Goal Question Metric approach. In *Encyclopedia of Software Engineering*, pages 528–532. John Wiley & Sons, New York.

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Child, M., Rosner, P., and Counsell, S. (2019). A comparison and evaluation of variants in the coupling between objects metric. *Journal of Systems and Software*, 151:120–132.

Dey, T., Mousavi, S., Ponce, E., Godfrey, T., Padhy, B., Nguyen, A., and Mockus, A. (2020). Detecting and characterizing bots that commit code. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR 2020)*, pages 209–219, Seoul, Republic of Korea. ACM.

El Emam, K., Benlarbi, S., Goel, N., and Rai, S. N. (2001). The confounding effect of class size on the validity of object-oriented metrics. *IEEE Transactions on Software Engineering*, 27(7):630–650.

Git (2026). git-rev-list documentation. `https://git-scm.com/docs/git-rev-list`. Opção `--first-parent`. Consultado em 11 de outubro de 2026.

GitHub (2021). Introducing GitHub Copilot: your AI pair programmer. `https://github.blog/news-insights/product-news/introducing-github-copilot-ai-pair-programmer/`. Publicado em 29 de junho de 2021.

Godfrey, M. W. and Tu, Q. (2000). Evolution in open source software: a case study. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 131–142. IEEE.

Hassan, A. E. (2008). The road ahead for Mining Software Repositories. In *2008 Frontiers of Software Maintenance*, pages 48–57. IEEE.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., and Wang, H. (2024). Large language models for software engineering: a systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8):1–79.

ISO/IEC (2011). Systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models. Interna-

tional Standard ISO/IEC 25010:2011, International Organization for Standardization, Geneva.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR '14)*, pages 92–101, Hyderabad, India. ACM.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates.

Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., and Jiang, Z. M. (2023). GitHub Copilot AI pair programmer: asset or liability? *Journal of Systems and Software*, 203:111734.

Mozannar, H., Bansal, G., Fourney, A., and Horvitz, E. (2024). Reading between the lines: modeling user behavior and costs in AI-assisted programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–18. ACM.

Munaiah, N., Kroh, S., Cabrey, C., and Nagappan, M. (2017). Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6):3219–3253.

Oliveira, J., Viggiato, M., Pinheiro, D., and Figueiredo, E. (2021). Mining experts from source code analysis: an empirical evaluation. *Journal of Software Engineering Research and Development*, 9(1):1–16.

Park, R. E. (1992). Software size measurement: A framework for counting source statements. Technical Report CMU/SEI-92-TR-020, Software Engineering Institute, Carnegie Mellon University.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. (2023). The impact of AI on developer productivity: evidence from GitHub Copilot. arXiv:2302.06590.

Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, 9 edition.

Radjenović, D., Heričko, M., Torkar, R., and Živković, A. (2013). Software fault prediction metrics: a systematic literature review. *Information and Software Technology*, 55(8):1397–1418.

Robbes, R., Matricon, T., Degueule, T., Hora, A., and Zacchiroli, S. (2026). Agentic much? Adoption of coding agents on GitHub. arXiv:2601.18341v2. Versão de 8 de abril de 2026. Consultado em 11 de outubro de 2026.

Singh, Y., Kaur, A., and Malhotra, R. (2010). Empirical validation of object-oriented metrics for predicting fault proneness models. *Software Quality Journal*, 18(1):3–35.

Spring (2014). Spring Boot 1.0 GA released. `https://spring.io/blog/2014/04/01/spring-boot-1-0-ga-released/`. Publicado em 1 de abril de 2014.

Sun, P., Kim, D.-K., Hua, M., and Lu, L. (2022). Measuring impact of dependency injection on software maintainability. *Computers*, 11(9):141.

Yetistiren, B., Ozsoy, I., Ayerdem, M., and Tuzun, E. (2023). Evaluating the code quality of AI-assisted code generation tools: an empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778.

Apêndice A: Artefatos, curadoria local e reprodução do painel

Os artefatos empíricos deste estudo partem da base de descoberta `Instrumentos/Codigos/Artefatos/selecao_spring_boot/runs/20260920_java/population.csv` (95.894 registros únicos) e encontram-se consolidados de forma modular e auditável em `pipeline_oficial_tcc/`, preservando separadamente a execução histórica congelada de 15/10/2026 (`Instrumentos/Codigos/Artefatos/evolucao_spring/runs/20261015_base_original_ate2017/`, que empregava corte móvel de três anos e 540 candidatos). No protocolo vigente, a janela `pushed_at >= 2022-01-01T00:00:00Z` sem filtro por número de estrelas seleciona 52.419 repositórios Java ativos em E4 e 785 candidatos criados até 2017, sem sorteio ou substituição de identificadores.

A etapa de higienização local nos quatro critérios conjuntos (C1 de finalidade não didática, C2 ajustado sem penalizar `demo`/`sample`/`example`, C3 de automação de *build* Maven/Gradle e C4 de presença de testes automatizados) é executada por `pipeline_oficial_tcc/03_higienizacao_reporeaper/reporeaper_curation.py`, gravando decisões individuais, *flags* lexicais e indicadores estruturais para os 785 candidatos (607 aprovados). Em seguida, `pipeline_oficial_tcc/04_extracao_ck/executar_extracao_ck.py` aplica o filtro de completude longitudinal com extração válida do CK em todas as eras previstas por grupo, materializando os 470 repositórios válidos finais em `repositorios_470_validos.csv` e os 1.427 *snapshots* em `measurements_470_validos.csv` (17 pioneiros em E1–E4, com 68 *snapshots*, e 453 principais em E2–E4, com 1.359 *snapshots*).

Cada linha de medição registra identificador, nome completo, contagem de estrelas, grupo temporal, era, SHA avaliado, status de extração, total de classes, arquivos Java distintos, LOC físico contextual (contado uma única vez por arquivo, sem duplicação por classes internas), as sete médias estruturais do CK (mantendo produção e testes discriminados nas extrações brutas) e a classificação temporalizada de vestígios de IA em E4 (`resultado_deteccao_ia_470_validos.csv`). Falhas de clonagem, ausência de *commit* na era ou ausência de classes Java permanecem documentadas nos artefatos de triagem sem conversão em zero, e classes com LCOM* indefinido são omitidas apenas do respectivo denominador.

Por fim, `pipeline_oficial_tcc/06_tabelas_artigo/gerar_tabelas_artigo.py` gera deterministicamente as tabelas LaTeX em `Artigo/Generated/` diretamente a partir dos CSVs validados ($n = 17$ em E1–E4; $n = 453$ em E2–E4; e $N = 470$ em E4, com $n_a = 75$ e $n_b = 395$), acompanhadas de manifestos JSON com *hashes* SHA-256 para verificação independente.